

The Math Behind It All: A Formula Tour of Surgical 3D Gaussian Splatting

Think of this as the engine-room tour. Every paper in your project is really just "take the base 3DGS engine, then bolt on one specific fix for one specific surgical problem." Once you see the base engine clearly, every paper's contribution becomes *one new equation added on top* — not a whole new universe to learn.

I've organized this by **problem**, not by paper, because that's how the ideas actually relate to each other. Each section names the paper the formula came from, so you always know exactly where to go if you want the full derivation.

Quick map:

#	Problem	Who solves it
0	The base engine everyone builds on	(shared, restated in every paper)
1	Tissue moves over time — how do you track it?	Deform3DGS
2	Gaussians "float" instead of hugging the tissue surface	Gaussian Pancakes, EndoGS
3	You only have one narrow viewpoint — where does 3D shape come from?	Gaussian Pancakes, EndoGS, Endo-E2E-GS, Endo4DGX, EndoNeRF (via review)
4	Tools block the view	EndoGS, Endo-E2E-GS (and a surprising counter-finding)
5	You don't want to depend on SfM/COLMAP for camera poses	FreeSurGS
6	Training a fresh model per video is slow — can you skip that?	Endo-E2E-GS
7	Lighting is inconsistent or bad	Endo-4DGX
8	You want to ask the scene questions in English	SurgTPGS
9	The older, pre-Gaussian way of doing all this	EndoSurf

0. The base engine (every paper starts here)

Every single paper opens its methods section by restating this exact machinery, because it's the shared "physics engine" underneath everything else.

What is a Gaussian, mechanically? Forget "3D Gaussian Splatting" as a scary phrase — it's just: *fill the scene with thousands of small, fuzzy, colored, stretchy blobs of fog, then blend them together to make a picture*. Each blob has:

- a center position $\mu \in \mathbb{R}^3$ (where is the blob)
- a color c (what color is it — sometimes stored as raw RGB, sometimes as spherical harmonics so it can shift with viewing angle)
- a shape, defined by a covariance matrix Σ (how stretched/squashed/tilted the blob is)
- an opacity $\alpha \in [0, 1]$ (how see-through it is)

The "fuzziness" itself — how much density the blob has at a 3D point x — is just a multivariate Gaussian bell curve:

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

You never let Σ be a free-for-all matrix (it has to stay a valid "ellipsoid shape"), so every paper builds it from a rotation R and a scale S :

$$\Sigma = R S S^T R^T$$

That's it — that's the "shape" of the blob: stretch it by S along its own axes, then rotate it by R .

Rendering = squashing 3D blobs onto your 2D camera and blending them. Given the camera's transform W and the projection Jacobian J :

$$\mu' = W\mu, \quad \Sigma' = JW\Sigma W^T J^T$$

Then, for a pixel, you walk through every blob that overlaps that pixel, **front to back**, and blend colors using the classic "alpha over" compositing rule (same idea as layering semi-transparent stickers):

$$C(x) = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Read that product term as "how much light survived getting through all the blobs in front of this one." A depth map falls out for free by swapping color c_i for depth d_i (each blob's distance from camera):

$$D(x) = \sum_{i=1}^N d_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Everything trainable boils down to one parameter set per Gaussian, usually written $\Theta = \{\mu, \Sigma, \alpha, c\}$ or spelled out as $g = \{\mu, c, r, s, \alpha\}$ (r = rotation quaternion, s = scale vector — the raw ingredients of Σ). Every paper below is really just: *"here's an extra term we add to how one of these five things gets computed or supervised."*

1. Problem: tissue deforms over time — how do you give each blob a "motion memory"?

Deform3DGS's answer: give every Gaussian its own tiny motion curve, built from overlapping bumps.

Instead of one giant neural network predicting motion for the whole scene (slow), Deform3DGS gives each Gaussian a handful of learnable "bump" functions — literally 1D Gaussian curves, but over *time* instead of space:

$$\tilde{b}(t; \theta, \sigma) = \exp\left(-\frac{1}{2\sigma^2}(t - \theta)^2\right)$$

Picture this as a little hill centered at time θ , with width σ . A single bump only "activates" near its own moment in time.

Then you sum a handful of these bumps together (each with its own learnable weight ω) to build a full motion curve over the whole video:

$$\psi(t; \Theta) = \sum_{j=1}^B \omega_j \tilde{b}(t; \theta_j, \sigma_j)$$

And this motion curve gets added directly onto the Gaussian's position, rotation, and scale:

$$\mu' = \mu + \psi(t; \Theta^\mu), \quad r' = r + \psi(t; \Theta^r), \quad s' = s + \psi(t; \Theta^s)$$

Why this is clever: because each bump only "fires" near its own time window, a big motion event at second 3 doesn't drag around the shape of the curve at second 20 — unlike Fourier/polynomial bases, where every coefficient influences the *entire* curve globally. That locality is exactly why it can represent sudden, localized tissue-tool interactions without corrupting the rest of the timeline.

A side quest — where do the Gaussians even start from (before any deformation)?

Deform3DGS reprojects each frame's masked depth into 3D:

$$P_i = K^{-1} D_i (I_i \odot M_i)$$

and to avoid holes wherever the frame-0 tool mask blocked something, it fuses in points from *other* frames wherever there's motion or missing coverage:

$$F = 1(|C_0 - \frac{1}{N} \sum_j C_j| > \tau) \cup (1 - M_0)$$

— "grab extra points anywhere the color differs a lot from the average frame, or anywhere frame 0's mask blocked the tool." That's the Motion-Aware Point Fusion (MAPF) step.

2. Problem: Gaussians drift into floating blobs instead of hugging the actual tissue surface

This is the "why does my reconstruction look like fuzzy clouds instead of a smooth organ" problem. Two different fixes:

Gaussian Pancakes' fix: force each Gaussian's "flat direction" to match the real surface normal.

First, estimate the *true* surface normal at each point the boring-but-reliable way — PCA on nearby points. Take the 10 nearest neighbors, center them, build their covariance:

$$C = \frac{1}{k-1} \sum_{j=1}^k n'_j n_j'^T, \quad k = 10$$

The eigenvector with the *smallest* eigenvalue is the normal (intuition: that's the direction the points *don't* spread out in — i.e., "flat").

Now compare that to the Gaussian's own shape: since a Gaussian is an ellipsoid, its "thinnest" axis (smallest scale value) should, if it's lying flat on the surface, point in the same direction as the real normal:

$$n_{GP} = R e_{min}$$

Then just penalize disagreement with cosine similarity:

$$L_G = 1 - \left| \frac{A \cdot B}{\|A\| \|B\|} \right|$$

This is literally what "pancaking" means in the paper's name — it's squashing each Gaussian flat against the surface, like a pancake, instead of letting it stay a floating blob.

Paired with a Huber-style depth loss (robust to outliers — behaves like squared error for small mistakes, linear for big ones so outliers don't dominate):

$$L_D(i) = \begin{cases} 0.5 \Delta D_i^2 & |\Delta D_i| < \delta \\ \delta(\Delta D_i - 0.5\delta) & \text{otherwise} \end{cases}, \quad \Delta D_i = |D_i - \hat{D}_i|$$

everything combines into one weighted total:

$$L = (1 - \lambda_1) L_{Image} + \lambda_1 L_{D-SSIM} + \lambda_2 L_D + \lambda_3 L_G$$

EndoGS's fix: derive an actual signed-distance function from the Gaussian density itself.

Different trick, same goal. If Gaussians are properly flat and opaque, the density function near the surface behaves almost like a clean signed-distance function (positive outside, zero at the surface, negative inside). EndoGS makes that approximation explicit:

$$f(p) = \pm s_{g^*} \sqrt{-2 \log(\bar{d}(p))}$$

(where g^* is the nearest Gaussian, s_{g^*} its smallest scale.) Then it just checks: does this implied surface distance agree with what you'd expect from the measured depth map? And does the implied normal (gradient of f) agree with the Gaussian's own flat axis?

$$L_{SDF} = \frac{1}{|P|} \sum_{p \in P} |\hat{f}(p) - f(p)|, \quad L_{norm} = \frac{1}{|P|} \sum_{p \in P} \left\| \frac{\nabla f(p)}{\|\nabla f(p)\|_2} - n_{g^*} \right\|_2^2$$

plus a small nudge encouraging Gaussians to just commit to being fully opaque rather than sitting at some wishy-washy in-between transparency (cross-entropy pulls values toward 0 or 1):

$$L_{opacity}(i) = - \sum_j (\sigma + \Delta\sigma)_j \log(\sigma + \Delta\sigma)_j$$

All stacked into one final loss:

$$L(i) = L_{L1}(i) + \lambda_D L_D(i) + \lambda_{TV1} L_{tv-spatial}(i) + \lambda_{TV2} L_{tv-temporal} + \lambda_S (L_{SDF} + L_{norm} + 0.5 L_{opacity}(i))$$

3. Problem: you've only got a stereo pair (or one viewpoint) — where does "3D-ness" even come from?

In normal 3D vision, you'd triangulate from many different camera angles. Endoscopy almost never gives you that — the scope barely moves, or moves only along a narrow track. So **every single paper substitutes a depth estimate** for true multi-view geometry, then treats that depth as a supervision signal (not a ground truth to blindly trust — a *target* to render toward).

Once you have a depth value D , comparing it to the rendered depth $\hat{D}$ is basically always some flavor of L1/Huber:

- Gaussian Pancakes / EndoGS: Huber loss on raw depth difference (shown above)
- Endo-E2E-GS: plain mean absolute error, $L_{depth} = \frac{1}{N} \sum_i |D_i - \hat{D}_i|$
- Endo4DGX: normalizes both depths first so the loss isn't dominated by scale,

$$L_{depth} = \lambda_{depth} \left| \frac{D}{D_{max}} - \frac{\hat{D}}{\hat{D}_{max}} \right| \odot M + \lambda_{tv} L_{tv}(D)$$

A neat trick from EndoNeRF (summarized in your review paper): don't sample rays uniformly — bias your attention toward where the depth map says the surface actually is. Rather than shooting a ray and checking random points along it, weight your samples with a Gaussian centered on the known depth:

$$\delta(s; u, v, i) = \exp \left(- \frac{(s - D_i(u, v))^2}{2\xi^2} \right)$$

— i.e., "spend your compute near where you already think the tissue is, not scattered along the whole ray."

Where does that depth number even come from in the first place? Usually stereo matching: given the known focal length f and stereo baseline b (the physical distance between the two lenses), pixel disparity d converts straight to metric depth via simple similar-triangles geometry, $D = fb/d$ (this is exactly why "paired image" / stereo pairs matter — no pair, no disparity, no depth).

4. Problem: surgical tools are constantly blocking tissue — how do you stop that from corrupting the model?

The blunt-instrument fix, used almost everywhere: **multiply the loss by a mask so tool pixels contribute zero gradient**. EndoGS's version:

$$L_{L1}(i) = |I_i \odot V_i - \hat{I}_i \odot V_i|$$

where V_i is a visibility mask (tissue = 1, tool = 0). A spatial total-variation term on the *masked* region keeps colors from drifting into weird patches right at the mask boundary:

$$L_{tv-spatial}(i) = TV(\hat{I}_i \odot M_i)$$

The interesting twist — masking has a real cost, and it shows up quantitatively. EndoE2E-GS explicitly tested training *with* vs. *without* masks and found masking dropped PSNR from 35.80 → 20.02, because excluding pixels from the loss also excludes them from *supervision* — you lose real information, not just noise. So the field is genuinely torn: mask to avoid corrupting geometry with tool pixels, or don't mask to keep more supervision signal. Neither choice is free.

5. Problem: you don't want SfM/COLMAP guessing your camera poses (because on tissue, it fails)

FreeSurGS's answer: estimate pose by matching where 3D points should move against where optical flow says they did move.***

Start with a naive guess — assume the camera keeps moving the way it's been moving (constant velocity):

$$\hat{q}_{t+1} = \hat{q}_t + (\hat{q}_t - \hat{q}_{t-1}) + \Delta q$$

Ordinary photometric loss (match rendered color to real color) is the standard way to refine a guessed pose:

$$L_{rgb} = (1 - \lambda)L_1 + \lambda L_{D-SSIM}$$

...but on textureless, shiny, wet tissue, this alone gets fooled easily (there's nothing distinctive to match). So FreeSurGS adds a second, independent signal: take the Gaussians you already have, and ask "if the camera really moved to pose $\hat{T}_{t+1}$, where would each point *appear* to move on screen?" — that's the **projection flow**:

$$\hat{f}_t = \hat{x}_{t+1} - x_t = \pi(\hat{T}_{t+1}^{-1}, X_t, K) - x_t, \quad X_t = \pi^{-1}(\hat{T}_t, \hat{D}_t(x_t), K)$$

Compare that to the *actual measured* optical flow (from RAFT) between the two frames, after filtering out unreliable/unbuilt regions with a visibility check:

$$M_v = \sum_i \alpha_i \prod_{j < i} (1 - \alpha_j) > \gamma$$

and a geometric consistency check (does the correspondence actually sit on its epipolar line?). What survives both filters becomes the flow loss:

$$L_{flow} = \|M \odot (\hat{f}_t - f_t)\|_2^2$$

Now pose estimation becomes a tug-of-war between "match the colors" and "match the physically-implied motion":

$$\hat{T}_{t+1} = \arg \min_{T_{t+1}} \lambda_1 L_{rgb} + \lambda_2 L_{flow}$$

and once the pose is locked in, the Gaussians themselves get updated the same way, with a depth term added in:

$$\hat{\Theta} = \arg \min_{\Theta} \lambda_1 L_{rgb} + \lambda_2 L_{flow} + \lambda_3 L_{depth}$$

The intuition in one sentence: optical flow doesn't care whether the surface has texture — it tracks *pixels*, so it survives exactly the conditions (uniform pink tissue, glossy highlights) that break feature-matching-based SfM.

6. Problem: training a new model per video is slow — can a network just *predict* the Gaussians directly?

Endo-E2E-GS's answer: yes, if you build a proper stereo pipeline first, then attach small CNN heads.

Step 1 — extract features from both stereo images with a shared encoder:

$$F_L = \phi(I_L; \theta_d), \quad F_R = \phi(I_R; \theta_d)$$

Step 2 — build a cost volume: for every candidate disparity d , how well does the left feature at (u, v) match the right feature shifted by d ?

$$C(u, v, d) = F_L(u, v)^T F_R(u - d, v)$$

Step 3 — refine that noisy cost volume into a clean disparity estimate iteratively, using a ConvGRU (a small recurrent refiner that "looks again" a few times):

$$d_{t+1} = \Gamma(d_t, C(u, v, d_t))$$

Step 4 — convert disparity to depth, reproject every pixel into 3D using that depth to get μ , sample color directly from the image for c (assuming roughly matte/Lambertian tissue), and let small CNN heads predict the remaining Gaussian attributes (r, s, α) straight from the depth+RGB features. No per-scene optimization loop at all — one forward pass, done.

Everything trains under one combined objective:

$$L_{total} = w_1 L_{depth} + w_2 L_{recon}, \quad L_{recon} = \alpha(1 - SSIM(I, \hat{I})) + \beta \cdot MAE(I, \hat{I})$$

Why this matters: every other method in this list re-optimizes thousands of Gaussian parameters from scratch for *every new video*. This one learns a general-purpose function once, then just runs it — the tradeoff being it can't refine itself to a specific scene the way per-scene optimization can.

7. Problem: lighting is inconsistent — dark corners, blown-out highlights, moving light source

Endo-4D-GX's answer: give the model an explicit "lighting condition" signal, then let it learn two separate correction passes — one local, one global.

First, cheaply classify each frame as bright or dark by comparing it to a lightness-prior estimate:

$$IC(I) = \begin{cases} \text{Bright,} & \text{mean}(I) > \text{mean}(p) \\ \text{Dark,} & \text{mean}(I) \leq \text{mean}(p) \end{cases}$$

Give every training image its own learnable "lighting fingerprint" embedding e (just a small learned vector, one per frame/view).

Local correction (per-Gaussian, small regions): a tiny network reads the embedding and outputs a brightness/contrast adjustment, applied like classic film-style color grading:

$$c_{tone} = \beta \cdot c + \gamma, \quad \beta, \gamma = f_{region}(c, e)$$

Global correction (whole image, smoother): a second network reads the *same* embedding but produces a single smooth curve applied everywhere:

$$\tilde{C}_{tone} = C_{tone} + \delta \cdot C_{tone}(1 - C_{tone}), \quad \delta = f_{spatial}(e)$$

(that $C(1 - C)$ shape is a classic S-curve trick — it gently pulls midtones while leaving pure black/white mostly alone.)

Finally, a loss that just says "however this scene was lit, the *final* reconstructed brightness should land near a sane, consistent exposure level":

$$L_{Exposure} = \left\| \text{avgpool1d} \left(\frac{1}{3} \sum_{j \in RGB} C_j \right) - E \right\|^2, \quad E = 0.6 \text{ (chosen target)}$$

All three combine simply:

$$L = L_{color} + L_{depth} + L_{Exposure}$$

Why two separate correction networks instead of one? The paper explicitly found that training one shared network on both bright and dark cases at once caused convergence problems — the corrections needed for "too dark" and "too bright" pull in genuinely different directions, so it splits them into dedicated bright/dark networks.

8. Problem: you want to type "show me the gallbladder" and get a 3D answer

SurgTPGS's answer: distill 2D vision-language features into the Gaussians themselves, as an extra channel alongside color.

Step 1 — get a segmentation mask from SAM, then pull vision-language features (from CAT-Seg) only from inside that masked region, compressed down to something small enough to train efficiently:

$$\hat{F}(x) = \Phi_{\text{encode}}(V_{\text{CAT-Seg}}(I \odot M(x)))$$

Step 2 — since tissue deforms, the *semantic* feature attached to a Gaussian needs to deform too, not just its position. A small conv-based network predicts how much the feature should shift, gated by a sigmoid so it can smoothly turn the update on/off over time:

$$f' = f + g(\text{Concat}(f, t)) \cdot \beta, \quad \beta = \frac{1}{1 + \exp(-\delta \psi(t; \Theta^f))}$$

Step 3 — at query time, take your English phrase, embed it with CLIP as ϕ_{text} , and compare it against the rendered image-embedding ϕ_{img} *relative to a set of generic "junk" phrases* (like "object," "stuff," "texture") — this relative comparison is what stops the model from just saying "yes" to everything:

$$\text{score} = \min_i \frac{\exp(\phi_{\text{img}} \cdot \phi_{\text{text}})}{\exp(\phi_{\text{img}} \cdot \phi_{\text{text}}) + \exp(\phi_{\text{img}} \cdot \phi_{\text{canon}}^i)}$$

Threshold that score (0.4 in the paper) and you get a 3D segmentation mask purely from a text prompt.

A regional smoothness term keeps large uniform areas (like "the liver") from having noisy, inconsistent features pixel-to-pixel:

$$L_{RS} = \sum_{l \in \hat{F}} \sum_{(i,j) \in l} |F_{ij} - \text{mean}(F_l)|, \quad |l| > 1000 \text{ px}$$

and the full loss just adds this semantic supervision alongside the usual color/depth terms:

$$L = |C - \hat{C}| + \left| \frac{D - \hat{D}}{D \hat{D}} \right| + |F - \hat{F}| + \lambda(L_{tv}(C) + L_{tv}(D) + L_{tv}(F) + L_{RS})$$

9. The older paradigm: what EndoSurf did *before* Gaussian Splatting existed (for contrast)

EndoSurf is NeRF/SDF-based, not GS-based — useful to see because it clarifies what GS actually simplified.

Instead of one flat set of Gaussian parameters, EndoSurf uses **three separate neural networks (MLPs)**:

- a deformation field, mapping a point at time t back to a fixed "canonical" reference shape: $\Delta x = \Psi_d(x_o, t), x_c = x_o + \Delta x$
- an SDF field, which outputs a signed distance ρ (how far this point is from the true surface — the surface itself is just wherever this hits zero): $(\rho, f) = \Psi_s(x_c)$, surface $= \{x \mid \Psi_s(x) = 0\}$
- a radiance field, predicting color from position + viewing angle + surface normal

Rendering still looks like the same "accumulate along a ray" idea, but opacity is derived from the SDF value instead of a raw learned parameter — this NeuS-style formula converts "distance from surface" into "how opaque is this point," so points far from the true surface contribute almost nothing:

$$\alpha_i = \max \left(1 - \frac{1 + \exp(-\rho_i/s)}{1 + \exp(-\rho_{i+1}/s)}, 0 \right)$$

Why the field mostly moved away from this: those three MLPs have to be queried densely along every ray at every training step — that's the "hours of training, low FPS" cost the GS papers keep citing as their whole reason for existing. Gaussians replace "query a network at arbitrary points" with "look up a small fixed set of explicit blobs," which is what makes real-time rendering possible.

Notation cheat sheet (watch for these traps across papers)

The papers don't share notation cleanly — same symbol, different meaning, depending who's writing:

Symbol	Sometimes means	Where
σ	base opacity of a Gaussian	EndoGS
σ	width of a time-bump in a deformation curve	Deform3DGS, SurgTPGS
α	opacity (0-1, used in rendering)	everywhere
α, β	region-aware tone-correction coefficients	Endo4DGX (totally different role)
Θ	full set of trainable Gaussian parameters	3DGS preliminaries generally
$\Theta^\mu, \Theta^r, \Theta^s$	just the deformation-curve parameters for position/rotation/scale	Deform3DGS
M	tool/occlusion mask	most papers
M_v, M_r	visibility mask / rigid-correspondence mask (unrelated to tools)	FreeSurGS
$\psi(t; \cdot)$	the deformation curve itself	Deform3DGS, SurgTPGS
f	a semantic feature vector	SurgTPGS
$f(p)$	a signed-distance value at point p	EndoGS
$f_{region}, f_{spatial}$	correction networks (unrelated to the " f " above)	Endo4DGX

When you're cross-referencing two papers side by side, always check what a symbol means *in that paper's own preliminaries section* before assuming it matches what you just read elsewhere.

If any of these don't click, just ask about that one piece — happy to zoom into any single formula, redraw the intuition a different way, or trace through a concrete numeric example.